find the second greatest number in array without sorting in cpp

To find the **second greatest number in an array without sorting** in C++, you can do it in a single pass using two variables:

💡 Idea:

- Keep track of:
  - `max1` → largest element
  - `max2` → second largest element
- Traverse the array once and update these values.

✅ C++ Code:

```cpp
#include <iostream>
#include <climits>
using namespace std;

int secondGreatest(int arr[], int n) {
    if (n < 2) {
        cout << "Array must have at least 2 elements";
        return -1;
    }

    int max1 = INT_MIN;
    int max2 = INT_MIN;

    for (int i = 0; i < n; i++) {
        if (arr[i] > max1) {
            max2 = max1;
            max1 = arr[i];
        }
        else if (arr[i] > max2 && arr[i] != max1) {
            max2 = arr[i];
        }
    }
```

```cpp
    if (max2 == INT_MIN) {
        cout << "No second greatest element";
        return -1;
    }

    return max2;
}

int main() {
    int arr[] = {10, 5, 20, 8, 20};
    int n = sizeof(arr) / sizeof(arr[0]);

    int result = secondGreatest(arr, n);
    if (result != -1)
        cout << "Second greatest: " << result;
```

C++

⚡ Key Points:

- Time complexity: **O(n)** (single loop)
- No sorting used
- Handles duplicates (e.g., `{20, 20}` → no second greatest)

If you want, I can also show a version using vectors or handling edge cases differently 👍

explain each line of code also what is intmin

Let's break this down step by step so you understand **every line** and also what `INT_MIN` means.

🔍 Code Explanation (Line by Line)

C++

- Includes input/output functions like `cout` and `cin`.

C++

- This is important.
- It provides constants like `INT_MIN` and `INT_MAX`.

C++

- Allows you to use `cout`, `cin`, etc. without writing `std::`.

🔧 Function Definition

C++

- Function that:
  - Takes an array `arr`
  - Takes its size `n`
- Returns the **second largest number**

```
if (n < 2) {
    cout << "Array must have at least 2 elements";
```

C++

- If array has less than 2 elements:
  - You **cannot** find a second largest
  - So print error and return `-1`

C++

💡 What is `INT_MIN` ?

- `INT_MIN` = **smallest possible integer value**
- Usually: `-2147483648`

👉 Why use it?

- We want initial values **smaller than any number in array**
- So any number we encounter will replace it

🔁 Loop through array

C++

- Loop runs from index `0` to `n-1`
- We check every element

📌 Case 1: New largest element

```
if (arr[i] > max1) {
    max2 = max1;
```

C++

- If current element is **greater than max1**:
  - Old max1 becomes max2
  - Current value becomes new max1

👉 Example:

```
arr[i] = 20
max1 = 10
→ max2 = 10
→ max1 = 20
```

📌 Case 2: Second largest update

```cpp
else if (arr[i] > max2 && arr[i] != max1) {
```

‹/› C++

- If:
  - Value is greater than `max2`
  - AND not equal to `max1` (avoid duplicates)

👉 Then update `max2`

⚠️ Edge Case Check

```cpp
if (max2 == INT_MIN) {
    cout << "No second greatest element";
```

‹/› C++

- If `max2` is still `INT_MIN`, it means:
  - No valid second largest was found
  - Example: `{5, 5, 5}`

‹/› C++

- Return the second largest number

🚀 Main Function

C++

- Program starts here

C++

- Array with values

C++

- Calculates number of elements
- Example:
  - Total bytes / size of one element

C++

- Calls function and stores result

C++

- If valid result:
  - Print it

🧠 Summary (Simple)

- `max1` → largest number
- `max2` → second largest number
- Loop updates both without sorting
- `INT_MIN` → starting lowest possible value

⚡ Quick Example Trace

Array: `{10, 5, 20, 8}`

Step	max1	max2
10	10	-∞
5	10	5
20	20	10
8	20	10

👉 Result: **10**

If you want, I can also show a **dry run with your own input** or a **simpler beginner version**.

sure

Great 👍 let's do a **dry run step-by-step** so you can really *see* how it works.

Input:

```
C++
```

🔁 Initial Values

```
C++
max1 = INT_MIN  →  very small number
max2 = INT_MIN
```

🔍 Iteration Step-by-Step

👉 i = 0 → arr[i] = 12

- 12 > max1 → YES
  → update:

```
max2 = max1 → INT_MIN
max1 = 12
```

✅ Now:

```
max1 = 12
max2 = INT_MIN
```

👉 i = 1 → arr[i] = 35

- 35 > max1 → YES
  → update:

```
max2 = 12
max1 = 35
```

✅ Now:

```
max1 = 35
max2 = 12
```

👉 i = 2 → arr[i] = 1

- 1 > max1 → NO
- 1 > max2 → NO

❌ No change

```
max1 = 35
max2 = 12
```

👉 i = 3 → arr[i] = 10

- 10 > max1 → NO
- 10 > max2 → NO

❌ No change

```
max1 = 35
max2 = 12
```

👉 i = 4 → arr[i] = 34

- 34 > max1 → NO
- 34 > max2 → YES

→ update:

```
max2 = 34
```

✅ Now:

```
max1 = 35
max2 = 34
```

👉 i = 5 → arr[i] = 1

- 1 > max1 → NO
- 1 > max2 → NO

❌ No change

✅ Final Result:

```
max1 = 35
max2 = 34
```

👉 Second greatest = 34

🧠 Visual Summary

Element	max1	max2
12	12	-∞
35	35	12
1	35	12
10	35	12
34	35	34
1	35	34

⚠️ Important Edge Case

Input:

```C++
{5, 5, 5}
```

- All values are same
- Condition `arr[i] != max1` prevents duplicates

👉 Result:

```
No second greatest element
```

💡 Simple Way to Remember

- `max1` = biggest
- `max2` = second biggest
- Always:
  - Update both when new max found
  - Update only max2 otherwise

If you want, I can next:

- 🔷 Convert this into **vector version**
- 🔷 Show **interview trick explanation**
- 🔷 Or give you a **practice question like this**

Is this conversation helpful so far?